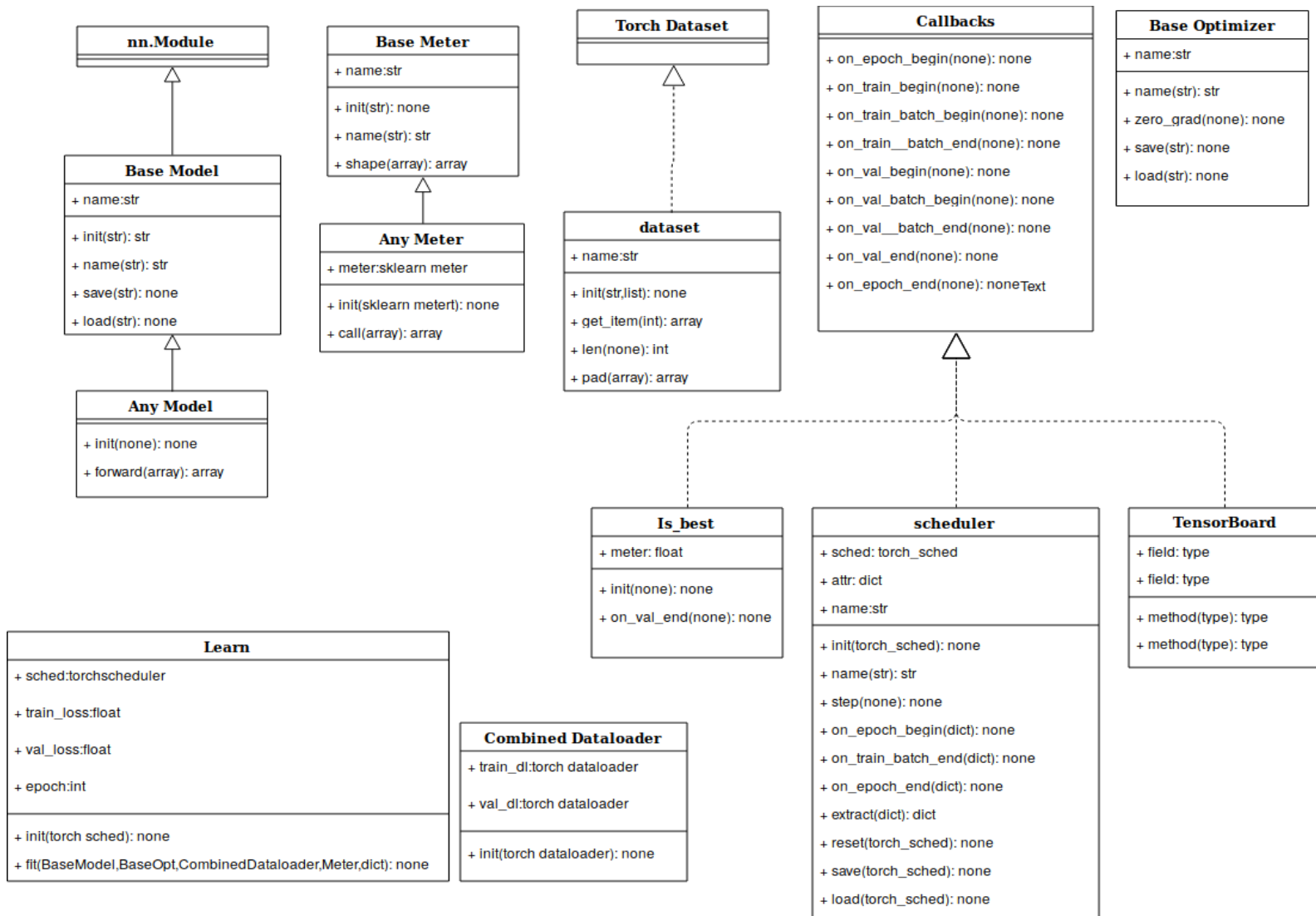


Software Design

Software overview:



The previous figure shows the different classes that are implemented in the software.

Our software consists of classes and methods we will pass by each class give an overview on it, explain only the unfamiliar components, Then we pass by other methods that don't belong to a class.

Base Model: located basemodel.py

Represents the base class for all customized modules

It provides the model with basic features as saving and loading its state dict

Note: Inputs provided from the dataset are in the type and shape needed for the loss function, So Models must:

1- Ensure consistency of Input to its components regarding type and dimensions

2- Return the output in the shape and type needed for the loss function.

Consists of:

1. `__init__`
2. `__name__`: Used in saving model by its name
3. `save`
4. `load`

Optimizer : located optimizer.py

Represent wrapper for torch optimizer class

Provides same interface of an optimizer and add some functionalities to it.

Consists of:

1. `__init__`
2. `__name__`
3. `zero_grad`
4. `step`
5. `save`
6. `load`

Scheduler : is located in scheduler.py

Represent a wrapper to torch schedulers, it implements the callback class.

Consists of:

1. `__init__`
2. `__name__`: Used for tag generation
3. `step`
4. `get_lr`
5. `on_epoch_begin`: Set the scheduler as the attribute of fit this helps in passing scheduler to the learner class.
6. `on_train_batch_end`
7. `on_epoch_end`
8. `extract`: Extracts numeric parameters of scheduler to reset it when needed
9. `reset` : Guarantee that at the end of the epoch the lr will be reset
This is important when using SGDR
10. `save`
11. `load`

dataset: located in data_handler.py

Implements Pytorch Dataset

Inputs:

`base_dir`: path to the dataset

`index(list)`: Indices of file in case not all data will be used

`dim(list,tuple,number)`: target dimension needed for the loss function

`mode(str)`: center or right padding

`word_type(type)`: to give the examples specific type needed in the loss function as float or int

Output:

Padded - reshaped single example or mini-batch

Consists of:

1. `__init__`
2. `__getitem__`
3. `__len__`
4. `pad`: Responsible for padding the input to fit required dimensions

CombineDataLoader: located in `data_handler.py`

Takes train dataloader and validation data loader and combine the in one object

Consists of:

1. `__init__`

Callbacks: located in `callbacks.py`

Base class class for all components that will use the callbacks concept

It contains different functions that represent different stages through the training and validation phase

Note: `**Kwargs` attribute must be added to any implemented method from the callbacks

Consists of:

1. `on_epoch_begin`
2. `on_train_begin`
3. `on_train_batch_begin`
4. `on_train_batch_end`
5. `on_val_begin`
6. `on_val_batch_begin`
7. `on_val_batch_end`
8. `on_val_end`
9. `on_epoch_end`

BaseMeter: located in `meter.py`

Base Class for metrics.

Provides the meters with `name` method to help in presenting result.

Consists of:

1. `__init__`
2. `shape`
 - Method check any inconsistency in type or shape of predicted and true values. Inconsistency occurs due to different representation needed among loss functions and meters
 - Inputs:
 - `pred(tensor)`: represent the predictions of the model
 - `true(tensor)`: represent the true values of the model
 - Output:
 - `pred`: 1D numpy array
 - `true`: 1D numpy array

accuracy: located in `meter.py`

Implementation for accuracy using `sklearn`.

Consists of:

1. `__init__`

2. `__call__`

f1 score: located in meter.py

Implementation for accuracy using sklearn.

Consists of:

3. `__init__`

4. `__call__`

Class Is_best: located in cbs.py

Search for the best model each epoch and save it if needed, implements callback class.

Consists of:

1. `__init__`

2. `on_val_end`

Class learner: located in fit.py

Class where training takes place, it has some arguments that help callbacks to reach the parameters of the model since sending and returning parameters through callbacks is very complicated.

Consists of:

1. `__init__`

2. `fit`

Methods

Split: Located in data_handler.py

Function: splits dataset to training and validation

Input:

`base_dir(str)`: path to the dataset

`prc(float)` : ratio between validation set size and dataset size

`dataset_pct(float)` :percentage of dataset to be used

Output:

train dataset

val dataset

Example:

`split('dataset/binary dataset/',dataset_pct=0.12,dim = (1,112,112), mode= 'center' ,word_type = torch.LongTensor)`

tqdm , clear_tqdm : Two methods located in fit.py

Responsible for the gui used to represent the progress of training

calc_metrics: located in meter.py

Method used to loop over multiple meters provided to fit function

gen_tag: located in utils.py

Generate a unique tag or name based on the model components

Could include time if needed to guarantee uniqueness of models

create_dir: located in utils.py

Creates directory in the given path and notify if directory already id writes in it

save_all: located in utils.py

Save all model details (model,optimizer,scheduler)

load_all: located in utils.py

Load all model details in place (model,optimizer,scheduler)

Classes and methods by files

basemodel.py

1. Class BaseModel

callbacks.py

1. Class Callbacks

cbs.py

1. Is_best

data_handler.py

1. Class dataset
2. Class combineDataLoader
3. split method

fit.py

1. Class learn

meter.py

1. ClassBaseMeter
2. Class accuracy
3. Class f1
4. calc_metrics method

optimizer.py

1. Class BaseOptimizer

scheduler.py

1. Class scheduler

utils.py

1. gen_tag method
2. create_dir method
3. save_all method
4. load_all method

None: Tensor board class is represented in the image but not implemented actually but it will use the same concept of callbacks and this is the intended UML representation to the class.